Programming in Python
IDC 212
Lecture - 9

October 11, 2025

Contents

1. Numpy

1.1 Axis
1.2 Arithmetic Operation
1.3 Matrix Multiplication
1.4 Broadcasting
1.5 Flatten
1.6 Stacking
1.7 Norms
1.8 Determinant
1.9 Inverse
1.10 Matrix Decompositions
1.11 Eigenvalues, Eigenvectors and Eigenfunctions

Numpy

Introduction to Tensor Shapes

- A **tensor** is a multi-dimensional array.
- Shape refers to the dimensions of the array.
- Axis indicates the direction along which operations like sum, mean, etc. are performed.

1D Array (Vector)

```python
import numpy as np
a = np.array([10, 20, 30])
print(a.shape)
```

Output: `(3,)`
Explanation: This is a 1D tensor with 3 elements. Only one axis (axis 0).

2D Array (Matrix)

```
b = np.array([[1, 2, 3],
              [4, 5, 6]])
print(b.shape)
```

Output: `(2, 3)`

Explanation: This is a 2D tensor with:

- Axis 0: 2 rows
- Axis 1: 3 columns

3D Tensor (Example)

```python
c = np.array([[[1, 2], [3, 4]],
              [[5, 6], [7, 8]]])
print(c.shape)
```

Output: `(2, 2, 2)`

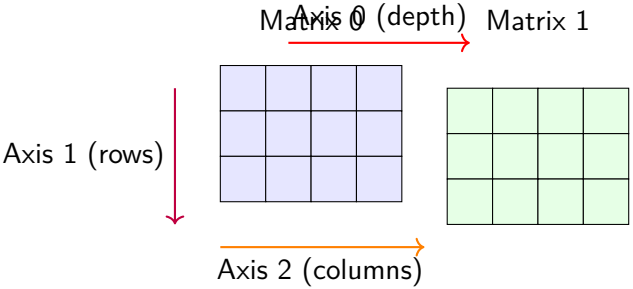
Explanation:

- Axis 0: 2 matrices
- Axis 1: 2 rows per matrix
- Axis 2: 2 columns per row

Visualizing 3D NumPy Array Shape: (2, 3, 4)

`np.zeros((2, 3, 4))` means:

- Axis 0 (depth): 2 matrices
- Axis 1 (rows): 3 rows per matrix
- Axis 2 (columns): 4 columns per row



Summary

- Shape defines the dimensions of the array.
- Axis 0 is usually the "rows", Axis 1 is "columns" in 2D.
- Axis increases with tensor rank: axis 2 for depth, etc.
- Understanding axes is key for operations like sum, mean, broadcasting.

Practice Question

What will be the output shape of the following array?

```
x = np.zeros((3, 4, 2))
```

- 3 blocks
- 4 rows per block
- 2 columns per row

Practice Question

What will be the output shape of the following array?

```
x = np.zeros((3, 4, 2))
```

Answer: `(3, 4, 2)`

- 3 blocks
- 4 rows per block
- 2 columns per row

Arithmetic Operations with Two Arrays

If we use another array instead of a scalar, the elements of both arrays will be component-wise combined:

```python
import numpy as np

A = np.array([[11, 12, 13],
              [21, 22, 23],
              [31, 32, 33]])
B = np.ones((3,3))

print("Adding to arrays: ")
print(A + B)

print("\nMultiplying two arrays: ")
print(A * (B + 1))
```

Output

Output:

```
Adding to arrays:
[[12. 13. 14.]
 [22. 23. 24.]
 [32. 33. 34.]]

Multiplying two arrays:
[[22. 24. 26.]
 [42. 44. 46.]
 [62. 64. 66.]]
```

Important Note

`A * B` in the previous example should not be mistaken for matrix multiplication. The elements are solely **component-wise multiplied**.

Matrix Multiplication

For this purpose, we can use the dot product. Using the previous arrays, we can calculate the matrix multiplication:

```
np.dot(A, B)
```

Output:

```
array([[36., 36., 36.],
       [66., 66., 66.],
       [96., 96., 96.]])
```

Definition of the Dot Product

The dot product is defined like this:
`np.dot(a, b, out=None)`

- For 2-D arrays: equivalent to matrix multiplication.
- For 1-D arrays: inner product of vectors.
- For scalars: simple multiplication.
- For N-D arrays: sum product over last axis of `a` and second-to-last of `b`.

Dot Product Parameters

Parameter	Meaning
`a`	array or array-like argument
`b`	array or array-like argument
`out`	optional parameter, must match the expected shape; improves performance

Examples of Using the Dot Product (1D & Scalars)

```python
print(np.dot(3, 4))        # 12

x = np.array([3])
y = np.array([4])
print(np.dot(x, y))        # 12

x = np.array([3, -2])
y = np.array([-4, 1])
print(np.dot(x, y))        # -14
```

Output:

```
12
12
-14
```

Examples of Using the Dot Product (2D Arrays)

```python
A = np.array([[1, 2, 3],
              [3, 2, 1]])
B = np.array([[2, 3, 4, -2],
              [1, -1, 2, 3],
              [1, 2, 3, 0]])

print(np.dot(A, B))
```

Output:

```
[[ 7  7 17  4]
 [ 9  9 19  0]]
```

Matrix Multiplication with np.dot

For two-dimensional arrays, `np.dot(A, B)` performs matrix multiplication.

```python
import numpy as np

A = np.array([[11, 12, 13],
              [21, 22, 23],
              [31, 32, 33]])
B = np.ones((3, 3))

R = np.dot(A, B)
print(R)
```

Output:

```
[[36. 36. 36.]
 [66. 66. 66.]
 [96. 96. 96.]]
```

Definition of the Dot Product

The dot product is defined as:
`np.dot(a, b, out=None)`

- Scalars: simple multiplication.
- 1D arrays: inner product.
- 2D arrays: matrix multiplication.
- N-D arrays: sum product over the last axis of `a` and the second-to-last of `b`.

Dot Product Parameters

Parameter	Meaning
`a`	array or array-like argument
`b`	array or array-like argument
`out`	optional parameter, must match expected shape; improves performance

Examples of Dot Product (Scalars & 1D)

```python
print(np.dot(3, 4))        # 12

x = np.array([3])
y = np.array([4])
print(np.dot(x, y))        # 12

x = np.array([3, -2])
y = np.array([-4, 1])
print(np.dot(x, y))        # -14
```

Output:

```
12
12
-14
```

Examples of Dot Product (2D Arrays)

```python
A = np.array([[1, 2, 3],
              [3, 2, 1]])
B = np.array([[2, 3, 4, -2],
              [1, -1, 2, 3],
              [1, 2, 3, 0]])

print(np.dot(A, B))
```

Output:

```
[[ 7  7 17  4]
 [ 9  9 19  0]]
```

Dot Product with 3D Arrays

```python
X = np.random.randint(1, 5, (3, 3, 3))
Y = np.random.randint(1, 5, (3, 3, 3))

R = np.dot(X, Y)

print("Shapes:")
print(X.shape, Y.shape, R.shape)
```

Output:

```
X.shape = (3, 3, 3)
Y.shape = (3, 3, 3)
R.shape = (3, 3, 3, 3)
```

The result is a 4D array due to contraction of axes.

Non-Symmetric 3D Dot Product

```python
X = np.random.randint(1, 5, (4, 2, 3))
Y = np.random.randint(1, 5, (2, 3, 5))

R = np.dot(X, Y)

print("Shapes:")
print(X.shape, Y.shape, R.shape)
```

Output:

```
X.shape = (4, 2, 3)
Y.shape = (2, 3, 5)
R.shape = (4, 2, 2, 5)
```

Arrays vs Matrices

Difference between 2D arrays and matrices:

```
A = np.array([[1, 2, 3],
              [2, 2, 2],
              [3, 3, 3]])
B = np.array([[ 3,  2,  1],
              [ 1,  2,  3],
              [-1, -2, -3]])

print(A * B)   # elementwise multiplication

MA = np.mat(A)
MB = np.mat(B)
print(MA * MB) # matrix multiplication
```

Note: Prefer `A @ B` in modern NumPy.

Logical Operators

We can apply logical operations elementwise on arrays using `np.logical_or` and `np.logical_and`:

```python
import numpy as np

a = np.array([[True, True],
              [False, False]])
b = np.array([[True, False],
              [True, False]])

print(np.logical_or(a, b))
print(np.logical_and(a, b))
```

Output:

```
[[ True  True]
 [ True False]]
[[ True False]
 [False False]]
```

Applying Operators on Arrays with Different Shapes

So far we have seen:

- Operator applied to an array and a scalar.
- Operator applied to two arrays of the same shape.

NumPy Broadcasting:

- If shapes differ, NumPy tries to stretch the smaller array.
- Rule: dimensions must match or one must be 1.
- Enables operations like adding vectors to matrices.

Example of Broadcasting

```
A = np.array([[1, 2, 3],
              [4, 5, 6]])
B = np.array([10, 20, 30])

print(A + B)
```

Output:

```
[[11 22 33]
 [14 25 36]]
```

Broadcasting

NumPy provides a powerful mechanism called **broadcasting**, which allows arithmetic operations on arrays of different shapes.

- The smaller array is virtually repeated to match the shape of the larger array.
- No actual copies are made, so broadcasting is memory efficient.
- Loops are executed internally in fast C code.

We demonstrate this with a simple example.

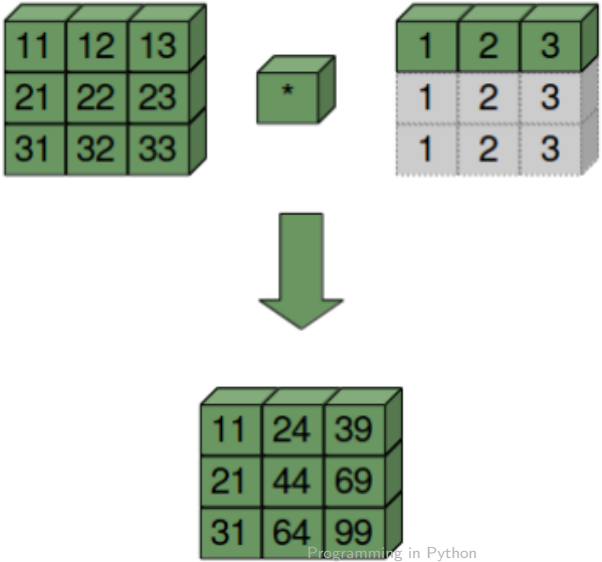
First Example of Broadcasting

```python
import numpy as np

A = np.array([[11, 12, 13],
              [21, 22, 23],
              [31, 32, 33]])
B = np.array([1, 2, 3])

print("Multiplication with broadcasting: ")
print(A * B)

print("Addition with broadcasting: ")
print(A + B)
```

Output:

```
Multiplication with broadcasting:
[[11 24 39]
 [21 44 69]
 [31 64 99]]
```

Visualization of Broadcasting



Second Example of Broadcasting

For this example, we first turn a row vector into a column vector:

```python
import numpy as np

B = np.array([1, 2, 3])
print(B[:, np.newaxis])
```

Output:

```
array([[1],
       [2],
       [3]])
```

Multiplication with Column Vector

Now we can multiply a matrix with this column vector using broadcasting:

```
A = np.array([[11, 12, 13],
              [21, 22, 23],
              [31, 32, 33]])

print(A * B[:, np.newaxis])
```

Output:

```
array([[11, 12, 13],
       [42, 44, 46],
       [93, 96, 99]])
```

Explanation of Broadcasting (Second Example)

Here `B[:, np.newaxis]` is treated as if it were:

$$\begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix} \quad \Rightarrow \quad \begin{bmatrix} 1 & 1 & 1 \\ 2 & 2 & 2 \\ 3 & 3 & 3 \end{bmatrix}$$

This expanded form allows elementwise multiplication with `A`.

Third Example of Broadcasting

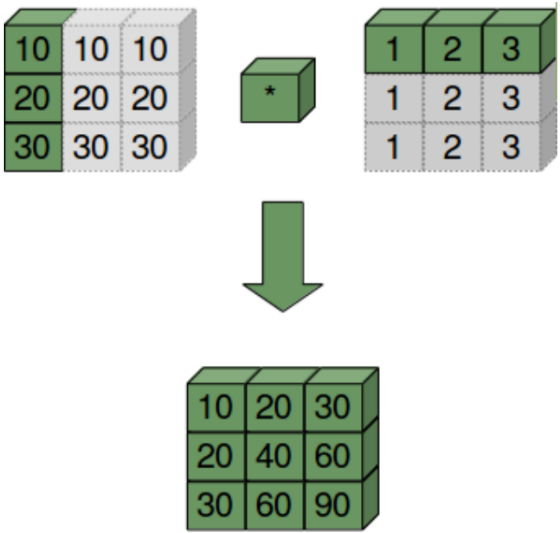
We can also use broadcasting with 1D arrays turned into column vectors:

```
A = np.array([10, 20, 30])
B = np.array([1, 2, 3])

print(A[:, np.newaxis])
```

Output:

```
array([[10],
       [20],
       [30]])
```

Visualization of Broadcasting



Flatten in NumPy

The method `flatten()` returns a copy of the array collapsed into one dimension. It accepts an optional keyword `order`, which can take values:

- `"C"` (default): C-style row-major order (last index varies fastest).
- `"F"`: Fortran-style column-major order (first index varies fastest).
- `"A"`: Preserve the array's original order (C/Fortran).

Example of Flatten

```python
import numpy as np

A = np.array([[[ 0,  1],
               [ 2,  3],
               [ 4,  5],
               [ 6,  7],
               [ 8,  9],
               [10, 11],
               [12, 13],
               [14, 15],
               [16, 17],
               [18, 19],
               [20, 21],
               [22, 23]]])

Flattened_X = A.flatten()
print(Flattened_X)
```

Output (default order "C"):

\scriptsize

Flatten with Different Orders

```python
print(A.flatten(order="C"))
print(A.flatten(order="F"))
print(A.flatten(order="A"))
```

Outputs:

```
\scriptsize
C order:
[ 0  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19 20 21 22 23]

F order:
[ 0  8 16  2 10 18  4 12 20  6 14 22  1  9 17  3 11 19  5 13 21  7 15 23]

A order:
[ 0  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19 20 21 22 23]
```

Notes on Flatten

- `flatten()` always returns a **copy** of the data.
- Compare with `ravel()`, which may return a **view** when possible.
- Choosing the right order (`"C"` or `"F"`) depends on how the array is stored or expected in memory.

Ravel in NumPy

The method `ravel()` returns a flattened one-dimensional array.

- Unlike `flatten()`, it returns a **view** of the array whenever possible.
- A copy is made only if needed.
- The order of flattening can be specified with the keyword `order`.

Available orders:

- `"C"`: C-style row-major order (last index varies fastest).
- `"F"`: Fortran-style column-major order (first index varies fastest).
- `"A"`: Use Fortran order if array is Fortran-contiguous, else C-order.
- `"K"`: Keep the order as stored in memory (except reversed if strides are negative).

Example of Ravel

```python
import numpy as np

A = np.array([[[ 0,  1],
               [ 2,  3],
               [ 4,  5],
               [ 6,  7],
               [ 8,  9],
               [10, 11],
               [12, 13],
               [14, 15],
               [16, 17],
               [18, 19],
               [20, 21],
               [22, 23]]])

print(A.ravel())
print(A.ravel(order="A"))
print(A.ravel(order="F"))
print(A.ravel(order="K"))
```

Output of Ravel

Outputs:

```
\scriptsize
C order (default):
[ 0  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19 20 21 22 23]

A order:
[ 0  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19 20 21 22 23]

F order:
[ 0  8 16  2 10 18  4 12 20  6 14 22  1  9 17  3 11 19  5 13 21  7 15 23]

K order:
[ 0  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19 20 21 22 23]
```

Notes: Flatten vs Ravel

- `flatten()` always returns a **copy**.
- `ravel()` returns a **view** when possible (more memory efficient).
- Both support `order="C"`, `"F"`, `"A"`, and `"K"`.
- Prefer `ravel()` when you only need a temporary 1D view of data.

Vector Stacking (Row vs Column Stack)

```python
import numpy as np

A = np.array([3, 4, 5])
B = np.array([1, 9, 0])

print(np.row_stack((A, B)))
print(np.column_stack((A, B)))
print(A.shape)
```

Output:

```
[[3 4 5]
 [1 9 0]]

[[3 1]
 [4 9]
 [5 0]]
```

Vector Stacking (Column Stack Multiple)

```
A = np.array([[3, 4, 5],
              [1, 9, 0],
              [4, 6, 8]])

print(np.column_stack((A, A, A)))
```

Output:

```
[[3, 4, 5, 3, 4, 5, 3, 4, 5],
 [1, 9, 0, 1, 9, 0, 1, 9, 0],
 [4, 6, 8, 4, 6, 8, 4, 6, 8]]
```

Vector Stacking (Column Stack with Rows)

```python
print(np.column_stack((A[0], A[0], A[0])))
```

Output:

```
[[3 3 3]
 [4 4 4]
 [5 5 5]]
```

Vector Stacking (Depth Stack)

```python
print(np.dstack((A, A, A)))
```

Output:

```
array([[[3, 3, 3],
        [4, 4, 4],
        [5, 5, 5]],

       [[1, 1, 1],
        [9, 9, 9],
        [0, 0, 0]],

       [[4, 4, 4],
        [6, 6, 6],
        [8, 8, 8]]])
```

Vector Norms

Definition: For $x \in \mathbb{R}^n$,

$$\|x\|_1 = \sum_{i=1}^{n} |x_i|, \quad \|x\|_2 = \sqrt{\sum_{i=1}^{n} x_i^2}, \quad \|x\|_\infty = \max_i |x_i|$$

Vector Norms in Python

Python Example:

```python
import numpy as np
x = np.array([1, -2, 3])

# Vector norms
np.linalg.norm(x, 1)       # 1-norm
np.linalg.norm(x, 2)       # 2-norm
np.linalg.norm(x, np.inf) # Infinity-norm
```

Matrix Norms

Definition: For $A \in \mathbb{R}^{m \times n}$,

- $\|A\|_F = \sqrt{\sum_{i,j} |a_{ij}|^2}$ (Frobenius norm)
- $\|A\|_1 = \max_j \sum_i |a_{ij}|$ (max column sum)
- $\|A\|_\infty = \max_i \sum_j |a_{ij}|$ (max row sum)
- $\|A\|_2 =$ largest singular value of A (spectral norm)

Matrix Norms in Python

Python Example:

```python
import numpy as np
A = np.array([[1, 2],
              [3, 4]])

np.linalg.norm(A, 'fro')  # Frobenius norm
np.linalg.norm(A, 1)      # 1-norm (max column sum)
np.linalg.norm(A, np.inf) # Infinity-norm (max row sum)
np.linalg.norm(A, 2)      # Spectral norm (largest singular
    value)
```

Important Remark

- For vectors:
$$\|x\|_2 = \sqrt{x_1^2 + x_2^2 + \cdots + x_n^2}$$
is the Euclidean length.
- For matrices:
$$\|A\|_2 = \sigma_{\max}(A)$$
where $\sigma_{\max}(A)$ is the largest singular value.
- Hence, **matrix 2-norm = spectral norm**.

Objective Question 1: Vector Norm

For $x = (3, -4)$, compute $\|x\|_2$.

A. 5

B. 7

C. 12

D. 25

Objective Question 1: Vector Norm

For $x = (3, -4)$, compute $\|x\|_2$.

A. 5

B. 7

C. 12

D. 25

Solution:

$$\|x\|_2 = \sqrt{3^2 + (-4)^2} = \sqrt{9 + 16} = \sqrt{25} = 5$$

Correct answer: **A**

Objective Question 2: Infinity Norm

For $x = (2, -7, 5)$, compute $\|x\|_\infty$.

A. 2

B. 5

C. 7

D. 14

Objective Question 2: Infinity Norm

For $x = (2, -7, 5)$, compute $\|x\|_\infty$.

A. 2

B. 5

C. 7

D. 14

Solution:

$$\|x\|_\infty = \max(|2|, |-7|, |5|) = 7$$

Correct answer: **C**

Objective Question 3: Matrix 1-Norm

Let

$$A = \begin{pmatrix} 1 & -2 \\ 3 & 4 \end{pmatrix}$$

Compute $\|A\|_1$.

A. 4

B. 6

C. 7

D. 10

Objective Question 3: Matrix 1-Norm

Let

$$A = \begin{pmatrix} 1 & -2 \\ 3 & 4 \end{pmatrix}$$

Compute $\|A\|_1$.

A. 4

B. 6

C. 7

D. 10

Solution:

$$\text{Col 1 sum} = |1| + |3| = 4, \quad \text{Col 2 sum} = |-2| + |4| = 6$$

$$\|A\|_1 = \max(4, 6) = 6$$

Correct answer: **B**

Objective Question 4: Matrix Infinity Norm

Let

$$B = \begin{pmatrix} 2 & -3 \\ -1 & 4 \end{pmatrix}$$

Compute $\|B\|_\infty$.

A. 5

B. 6

C. 7

D. 8

Objective Question 4: Matrix Infinity Norm

Let

$$B = \begin{pmatrix} 2 & -3 \\ -1 & 4 \end{pmatrix}$$

Compute $\|B\|_\infty$.

A. 5

B. 6

C. 7

D. 8

Solution:

$$\text{Row 1 sum} = |2| + |-3| = 5, \quad \text{Row 2 sum} = |-1| + |4| = 5$$

$$\|B\|_\infty = \max(5, 5) = 5$$

Correct answer: **A**

Objective Question 5: Frobenius Norm

For

$$C = \begin{pmatrix} 1 & 2 \\ -2 & 3 \end{pmatrix}$$

compute $\|C\|_F$.

A. $\sqrt{10}$

B. $\sqrt{15}$

C. $\sqrt{18}$

D. $\sqrt{20}$

Objective Question 5: Frobenius Norm

For

$$C = \begin{pmatrix} 1 & 2 \\ -2 & 3 \end{pmatrix}$$

compute $\|C\|_F$.

A. $\sqrt{10}$
B. $\sqrt{15}$
C. $\sqrt{18}$
D. $\sqrt{20}$

Solution:

$$\|C\|_F = \sqrt{1^2 + 2^2 + (-2)^2 + 3^2} = \sqrt{1 + 4 + 4 + 9} = \sqrt{18}$$

Correct answer: **C**

Determinant of a Square Matrix

```python
import numpy as np

# Define a 2x2 matrix
A = np.array([[4, 2],
              [3, 1]])

det_A = np.linalg.det(A)
print("Determinant of A:", det_A)
```

Output:

```
Determinant of A: -2.0
```

The determinant exists and is non-zero, so the matrix is invertible.

Case: Determinant = 0 (Singular Matrix)

```python
# Define a singular matrix
# (rows are linearly dependent)
B = np.array([[2, 4],
              [1, 2]])

det_B = np.linalg.det(B)
print("Determinant of B:", det_B)
```

Output:

```
Determinant of B: 0.0
```

The determinant is zero, so the matrix is singular and has no inverse.

Case: Determinant Not Defined (Non-Square Matrix)

```python
# Non-square matrix
C = np.array([[1, 2, 3],
              [4, 5, 6]])

# This will raise an error
det_C = np.linalg.det(C)
```

Error:

```
LinAlgError: Last 2 dimensions of the array
must be square
```

For non-square matrices, the determinant is not defined.

Inverse of an Invertible Matrix

```python
import numpy as np

A = np.array([[4, 7],
              [2, 6]])

det_A = np.linalg.det(A)
print("Determinant:", det_A)

A_inv = np.linalg.inv(A)
print("Inverse:\n", A_inv)

# Verify A * A_inv = I
print("Check:\n", np.dot(A, A_inv))
```

Output:

```
Determinant: 10.0
Inverse:
```

Case: Singular Matrix (No Inverse)

```
B = np.array([[2, 4],
              [1, 2]])

det_B = np.linalg.det(B)
print("Determinant:", det_B)

B_inv = np.linalg.inv(B)
```

Output:

```
Determinant: 0.0
LinAlgError: Singular matrix
```

Here $\det(B) = 0$, so B^{-1} does not exist.

Case: Non-Square Matrix (Inverse Undefined)

```
C = np.array([[1, 2, 3],
              [4, 5, 6]])

np.linalg.inv(C)
```

Error:

```
LinAlgError: Last 2 dimensions of the array
must be square
```

For non-square matrices, the inverse is not defined.

Matrix Decompositions in NumPy

NumPy provides several important matrix decompositions:

- **QR Decomposition**: $A = QR$
- **Eigen Decomposition**: $Av = \lambda v$
- **Singular Value Decomposition (SVD)**: $A = U\Sigma V^T$
- **Cholesky Decomposition**: $A = LL^T$

These are fundamental tools in linear algebra, data science, and machine learning.

QR Decomposition

Theory:

$$A = QR, \quad Q^T Q = I, \; R \text{ is upper triangular}$$

Code:

```python
import numpy as np

A = np.array([[1, 1, 0],
              [1, 0, 1],
              [0, 1, 1]])

Q, R = np.linalg.qr(A)
print("Q:\n", Q)
print("R:\n", R)
print("Check Q*R:\n", Q @ R)
```

Eigen Decomposition

Theory:

$$Av = \lambda v, \quad A = V\Lambda V^{-1}$$

Code:

```
vals, vecs = np.linalg.eig(A)
print("Eigenvalues:", vals)
print("Eigenvectors:\n", vecs)

# Verify Av = lambda*v for first eigenpair
print("Check:", A @ vecs[:,0],
                vals[0] * vecs[:,0])
```

Singular Value Decomposition (SVD)

Theory:

$$A = U\Sigma V^T$$

Code:

```python
U, S, Vt = np.linalg.svd(A)

print("U:\n", U)
print("Singular values:", S)
print("V^T:\n", Vt)

# Reconstruct A
Sigma = np.zeros_like(A, dtype=float)
np.fill_diagonal(Sigma, S)
print("Reconstructed:\n", U @ Sigma @ Vt)
```

Cholesky Decomposition

Theory: For symmetric positive-definite A:

$$A = LL^T, \quad L \text{ is lower triangular}$$

Code:

```
A = np.array([[4, 2],
              [2, 3]])

L = np.linalg.cholesky(A)
print("L:\n", L)
print("Check:\n", L @ L.T)
```

Summary of Decompositions

Decomposition	Form	Applications
QR	$A = QR$	Least squares, orthogonalization
Eigen	$A = V\Lambda V^{-1}$	PCA, control, spectral analysis
SVD	$A = U\Sigma V^T$	PCA, compression, pseudoinverse
Cholesky	$A = LL^T$	Optimization, covariance

Eigenvalues and Eigenvectors: Definition

For a square matrix $A \in \mathbb{R}^{n \times n}$:

$$Av = \lambda v$$

- v: eigenvector (non-zero vector)
- λ: eigenvalue (scalar)

Example:

$$A = \begin{bmatrix} 2 & 0 \\ 0 & 3 \end{bmatrix}, \quad v = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$$

$$Av = \begin{bmatrix} 2 \\ 0 \end{bmatrix} = 2v$$

Thus, $\lambda = 2$, $v = (1, 0)^T$.

Geometric Meaning

- Eigenvectors are **directions** preserved under transformation.
- Eigenvalues are the **stretching/shrinking factors**.
- $\lambda = 0$: vector is flattened (singular matrix).
- Complex eigenvalues: describe **rotations**.

Illustration:

$$A = \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix}$$

is a rotation by 90°. Its eigenvalues are $\pm i$.

Eigenfunctions (Operators on Functions)

Generalization from matrices to **operators** T:

$$Tf = \lambda f$$

Example: Derivative Operator

$$T = \frac{d}{dx}, \quad \frac{df}{dx} = \lambda f(x)$$

Solution:

$$f(x) = Ce^{\lambda x}$$

So exponential functions are eigenfunctions of $\frac{d}{dx}$ with eigenvalue λ.

Eigenfunctions: Laplacian Example

$$\nabla^2 f = \lambda f$$

Solutions: sines and cosines are eigenfunctions of the Laplacian. This is the foundation of Fourier series and PDE solutions.

In quantum mechanics: Schrödinger equation is an eigenvalue problem: Hamiltonian operator $\hat{H}\psi = E\psi$

- ψ: eigenfunction (wavefunction)
- E: eigenvalue (energy)

Applications of Eigenvalues/Eigenfunctions

- **Linear Algebra**: Principal Component Analysis (PCA).
- **Differential Equations**: Vibrations, heat conduction.
- **Quantum Mechanics**: Operators, wavefunctions, energy levels.
- **Graph Theory**: Spectral clustering, network connectivity.
- **Data Science**: Dimensionality reduction, embeddings.

Eigenvalues in NumPy

```python
import numpy as np

A = np.array([[2, 0],
              [0, 3]])

# Eigen decomposition
vals, vecs = np.linalg.eig(A)
print("Eigenvalues:", vals)
print("Eigenvectors:\n", vecs)

# Verify Av = lambda*v
print("Check:", A @ vecs[:,0],
              vals[0] * vecs[:,0])
```

Eigenvectors vs Eigenfunctions

Eigenvectors (finite-dimensional):

$$Av = \lambda v$$

- A: square matrix ($n \times n$) - v: eigenvector (finite list of numbers) - λ: eigenvalue (scalar)

Eigenfunctions (infinite-dimensional):

$$Tf(x) = \lambda f(x)$$

- T: linear operator (e.g., derivative, Laplacian) - $f(x)$: eigenfunction (function, not just vector) - λ: eigenvalue
Eigenfunctions are the *function-space analog* of eigenvectors.

Example: Derivative Operator

Operator:

$$T = \frac{d}{dx}, \quad \frac{df}{dx} = \lambda f(x)$$

Solve differential equation:

$$f(x) = Ce^{\lambda x}$$

- Exponential functions are eigenfunctions of $\frac{d}{dx}$.
- Eigenvalue is λ.

Example: Laplacian Operator

Consider the 1D Laplacian:

$$-\frac{d^2 f}{dx^2} = \lambda f(x), \quad f(0) = f(L) = 0$$

Solutions:

$$f_n(x) = \sin\left(\frac{n\pi x}{L}\right), \quad \lambda_n = \left(\frac{n\pi}{L}\right)^2, \quad n = 1, 2, 3, \ldots$$

- Eigenfunctions: sine functions.
- Eigenvalues: squares of integer multiples of π/L.
- Basis for Fourier series and PDE solutions.

How to Approximate Eigenfunctions in NumPy

- Operators are infinite-dimensional $\Rightarrow$ cannot directly compute in NumPy.
- Idea: **discretize the operator** using finite differences.
- Example: approximate second derivative by a matrix (discretized Laplacian).
- Then compute eigenvalues/eigenvectors of this matrix.
- The eigenvectors approximate the continuous eigenfunctions.

NumPy Example: Discretized Laplacian

```python
import numpy as np
import matplotlib.pyplot as plt
# Discretize [0,1] into n interior points
n = 50
h = 1.0 / (n+1)
# Finite-difference Laplacian matrix
L = -2*np.eye(n) + np.eye(n,1) + np.eye(n,-1)
L = L / (h**2)
# Eigen decomposition
vals, vecs = np.linalg.eigh(L)
print("First 3 eigenvalues:", vals[:3])
# Plot first 3 eigenfunctions (discretized)
x = np.linspace(0,1,n+2)[1:-1]
plt.plot(x, vecs[:,0], label="1st mode")
plt.plot(x, vecs[:,1], label="2nd mode")
plt.plot(x, vecs[:,2], label="3rd mode")
plt.legend(); plt.show()
```

Output and Interpretation

- Eigenvalues approximate $\pi^2, (2\pi)^2, (3\pi)^2, \ldots$.
- Eigenvectors approximate sine functions $\sin(n\pi x)$.
- Plot shows sinusoidal eigenmodes:
  - 1st eigenfunction: one half-wave (fundamental mode).
  - 2nd eigenfunction: one full sine wave.
  - 3rd eigenfunction: one and a half sine waves, etc.
- These discrete eigenvectors are the **numerical approximation** of eigenfunctions.

Thank You